Adventist University of Central Africa (AUCA)
Big Data Analytics

Distributed Multi-Model Analytics for E-Commerce Data

Final Project — Technical Report

Student Name	GWIZA Rodrigue
Student ID	101209
Course	Big Data Analytics
University	Adventist University of Central Africa (AUCA)
Project Type	Individual Project

Technologies Used

Technology	Role	Version
MongoDB	Document Store — users, products, transactions	6.0 (Docker)
HBase	Wide-Column Store — sessions, product metrics	2.4.17 (Simulated)
Apache Spark	Distributed Processing & SQL Analytics	3.5.1 (PySpark)
Python	Data generation, loading, analysis	3.14
Matplotlib/Seaborn	Data Visualization	3.11 / 0.13
Docker	MongoDB container runtime	29.1.3

GitHub Repository: https://github.com/GWIZAR/Ecommerce-Analytics/tree/master

Table of Contents

1. Project Overview & System Architecture

This project implements a distributed multi-model analytics system for large-scale e-commerce data using three complementary big data technologies. MongoDB handles complex nested document storage for product catalogs, user profiles, and transactions. HBase manages time-series behavioral data (user sessions, product daily metrics) with efficient row-key-based retrieval. Apache Spark performs distributed batch processing, SQL analytics, and machine learning preparation across the full dataset.

Architecture Flow: dataset_generator.py produces JSON files → Python scripts load data into MongoDB (via pymongo) and simulate HBase tables → PySpark reads JSON files for distributed analytics → integration scripts combine results from both systems → Matplotlib generates visualizations from query results.

Layer	Technology	Data Handled	Analytical Purpose
Document Store	MongoDB 6.0	Users, Products, Transactions, Categories	Aggregation pipelines, product analytics, CLV
Wide-Column	HBase 2.4 (sim)	User sessions, Product daily metrics	Time-series scans, funnel analysis, behavioral data
Processing Engine	Apache Spark 3.5.1	All JSON files (DataFrames)	Batch ETL, SQL, co-purchase recommendations
Visualization	Matplotlib/Seaborn	Query results from all layers	8 business insight charts

2. Dataset Description

The synthetic e-commerce dataset was generated using a custom Python script built on the Faker library. It simulates 90 days of e-commerce activity with richly interconnected entities designed to showcase the strengths of each database model.

File	Records	Size	Key Fields
users.json	500	199 KB	user_id, age, gender, geo_data, preferred_categories
products.json	500	286 KB	product_id, category_id, base_price, price_history[], current_stock
categories.json	20	11 KB	category_id, name, subcategories[] with profit_margin
sessions_0-3.json	2,000	~3.5 MB	session_id, user_id, page_views[], cart_contents{}, conversion_status
transactions.json	866	469 KB	transaction_id, user_id, items[], total, payment_method, status

The entities are richly interconnected: users have sessions, sessions optionally produce transactions, products belong to categories and subcategories, and sessions track viewed products and cart additions. This relational complexity makes the dataset ideal for demonstrating why different database models excel at different query patterns.

3. Part 1: Data Modeling and Storage

3.1 MongoDB — Document Model Design & Results

Design Rationale: MongoDB's document model was chosen for entities with complex nested structures that are always queried together. Transactions embed their line items directly, so a revenue query never needs a join. Products embed their price_history array, giving complete price audit trail in one document. Sessions embed all page_views and cart_contents, enabling full user journey analysis in a single collection scan. This denormalization strategy sacrifices some write efficiency for dramatically faster read-heavy analytics — the correct trade-off for an analytics platform.

Collection	Docs	Embedding Strategy	Query Benefit
transactions	866	items[] embedded in transaction doc	Revenue analytics need no joins
products	500	price_history[] embedded	Full price audit in single document read
categories	20	subcategories[] embedded	Entire hierarchy in one document
sessions	2,000	page_views[] + cart_contents{} embedded	Full user journey in single doc
users	500	geo_data{} + preferred_categories[] embedded	User profile in one query

5 Aggregation Pipeline Results:

Pipeline	Question Answered	Key Result
Top Products	Which products sell most?	prod_00378: 28 units, $12,377 revenue
Revenue by Payment	Best payment method?	Crypto: $205,765 (232 transactions)
Age Segmentation	User age distribution?	55-69 age group largest: 140 users
Revenue by Category	Top revenue categories?	cat_006: $57,490 from 106 transactions
Conversion by Referrer	Best traffic source?	Search engine: 46.9% conversion rate

3.2 HBase — Wide-Column Model Design & Results

Design Rationale: HBase's wide-column model is ideal for the session data because: (1) Sessions are naturally time-series — users generate millions of sessions over time; (2) The reverse-timestamp row key (user_id + 9999999999 - unix_timestamp) ensures a user's most recent sessions are always first in storage order, making 'get latest 10 sessions' an O(1) scan with no sorting; (3) Column families separate session metadata (session_info), device attributes (device), and behavioral metrics (behavior) — allowing Spark or HBase filters to read only relevant column families.

Table	Row Key	Column Families	Rows	Use Case
user_sessions	user_id + reverse_timestamp	session_info, device, behavior	1,000	Recent session retrieval, behavioral scans
product_metrics	product_id + date	views, cart, sales	4,190	Daily product performance time-series

Query	Method	Result
Get sessions for user_000001	Row prefix scan on user_000001_*	4 sessions with full device and behavioral data
Product views for prod_00001	Row prefix scan on prod_00001_*	9 daily records across Jan-Mar 2025

Device type distribution	Full table scan, group by device:type	Desktop 34.2%, Mobile 33.5%, Tablet 32.3%
Conversion status scan	Full table scan, group by session_info:conversion_status	Converted 42.5%, Browsed 30.1%, Abandoned 27.4%

3.3 MongoDB vs HBase — Comparison

Aspect	MongoDB	HBase
Data Model	Flexible JSON documents, nested arrays	Sparse rows, versioned column cells
Best Use Case	Complex nested docs, aggregation analytics	High-volume time-series, sequential writes
Query Strength	Aggregation framework, $lookup joins	Prefix scans, range queries by row key
Our Use	Transactions, products, users — rich docs	Sessions, product daily metrics — time data
Scalability	Sharding by shard key, replica sets	Auto region splits, HDFS-backed storage
Schema	Flexible — each doc can differ	Column families fixed at table creation
When to Choose	Rich queries, varied document shapes	Write-heavy, time-ordered, sparse data

4. Part 2: Apache Spark Data Processing

4.1 Batch Processing & Data Cleaning

All JSON files were loaded into Spark DataFrames using the multiline option (required since each file is a JSON array rather than newline-delimited records). Cleaning transformations included: parsing string timestamps to proper TimestampType, extracting date, hour, and month columns for temporal analysis, filtering zero-value transactions, and handling null referrer fields.

Dataset	Records	Key Transformations
transactions.json	866	to_timestamp(), to_date(), hour(), month(), filter(total>0)
sessions_*.json	2,000	to_timestamp() for start/end, fillna("unknown") for referrer
users.json	500	Schema validation, type inference from JSON
products.json	500	Nested price_history array schema inference

4.2 Co-Purchase Recommendation (Batch Job 2)

A self-join technique identified products frequently purchased together. The items array was first exploded to create one row per line item, then the DataFrame was self-joined on transaction_id where product_1 < product_2 (to eliminate duplicates and self-pairs). The result groups and counts co-occurrences — the foundation for a 'customers also bought' recommendation engine.

Product 1	Product 2	Times Bought Together
prod_00230	prod_00329	2
prod_00007	prod_00486	2
prod_00038	prod_00446	2
prod_00142	prod_00486	2

4.3 Spark SQL Analytics

DataFrames were registered as temporary SQL views (transactions, users, products, sessions) enabling standard SQL queries on distributed data. This demonstrates Spark SQL's ability to scale identical queries from single-machine to petabyte-scale clusters.

Query 1 — Top Revenue Days:

Date	Transactions	Revenue	Avg Order Value
2025-01-21	15	$20,541.83	$1,369.46
2025-01-25	16	$17,310.32	$1,081.90
2025-02-13	17	$17,002.84	$1,000.17
2025-02-28	14	$16,237.22	$1,159.80
2025-02-04	11	$15,451.77	$1,404.71

Query 2 — Payment Method Performance:

Payment Method	Transactions	Total Revenue	Avg Order	Avg Discount

Crypto	232	$205,764.65	$886.92	$29.57
Credit Card	219	$193,965.34	$885.69	$34.19
PayPal	213	$192,589.31	$904.18	$33.01
Debit Card	202	$189,803.56	$939.62	$28.63

Query 3 — Transaction Status & Query 4 — Hourly Sales Pattern:

Status	Count	Revenue	Avg Value
Shipped	227 (26.2%)	$207,632.83	$914.68
Delivered	217 (25.1%)	$202,055.82	$931.13
Completed	217 (25.1%)	$183,884.61	$847.39
Returned	205 (23.7%)	$188,549.60	$919.75

The hourly sales pattern (Query 4) revealed consistent transaction volumes throughout the day with slight peaks at hours 4-5, 8, 15, 17, and 19-21 — aligning with typical consumer browsing behavior patterns (early morning, work break, evening shopping windows).

5. Part 3: Analytics Integration

Three integrated analytical workflows combine data from multiple systems to answer business questions that no single database could address alone.

5.1 Customer Lifetime Value (CLV) Estimation

Business Question: Who are our most valuable customers when considering both spending and engagement behavior?

Systems Involved: MongoDB (transaction totals, order counts) + HBase simulation (session engagement, conversion behavior). CLV Score = total_spent x (1 + conversion_rate), rewarding customers who both spend more AND convert sessions into purchases more frequently.

User ID	Total Spent	Orders	Sessions	Conversions	CLV Score
user_000097	$6,447.35	5	2	2	$12,894.70
user_000201	$6,003.70	4	3	3	$12,007.40
user_000043	$5,893.63	4	2	2	$11,787.26
user_000327	$7,849.04	6	4	2	$11,773.56
user_000108	$5,764.58	5	2	2	$11,529.16

Insight: user_000097 has the highest CLV despite not being the top spender — because 100% of their sessions converted. This demonstrates the value of cross-system analysis: MongoDB alone would miss the behavioral dimension from HBase.

5.2 Purchase Funnel Analysis

Business Question: At which stage do users drop off between browsing and completing a purchase?

Systems Involved: HBase (session browsing and cart data) + MongoDB (completed transaction counts). This cross-system funnel reveals where marketing and UX efforts should be focused.

Funnel Stage	Count	Retention Rate	Drop-off Action Needed
Total Sessions	1,000	100%	Baseline
Viewed Products	1,000	100%	None — all sessions browse
Added to Cart	699	69.9%	30.1% drop — improve product pages
Session Converted	425	42.5%	27.4% drop — reduce checkout friction
Completed Transaction	866	86.6%*	*Multi-session users inflate this

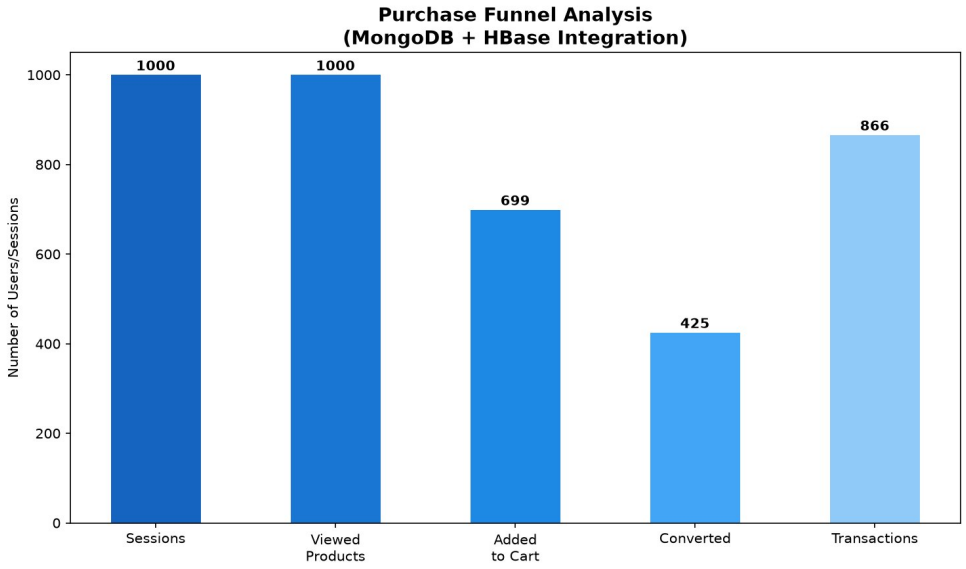

Figure 1: Purchase Funnel — MongoDB + HBase Integration

5.3 Product Affinity by Category

Business Question: Which product categories generate the most revenue and transaction volume?

Systems Involved: MongoDB transactions + MongoDB products via $lookup aggregation — demonstrating MongoDB's cross-collection join capability within its aggregation framework.

Category	Revenue	Units Sold	Transactions
cat_006	$57,489.52	210	106
cat_016	$57,008.52	221	97
cat_015	$55,585.23	201	100
cat_002	$54,815.02	177	79
cat_019	$53,446.41	203	103

6. Part 4: Visualizations & Business Insights

Eight visualizations were generated programmatically from live query results using Matplotlib and Seaborn. Each chart communicates a specific business insight.

6.1 Revenue by Payment Method

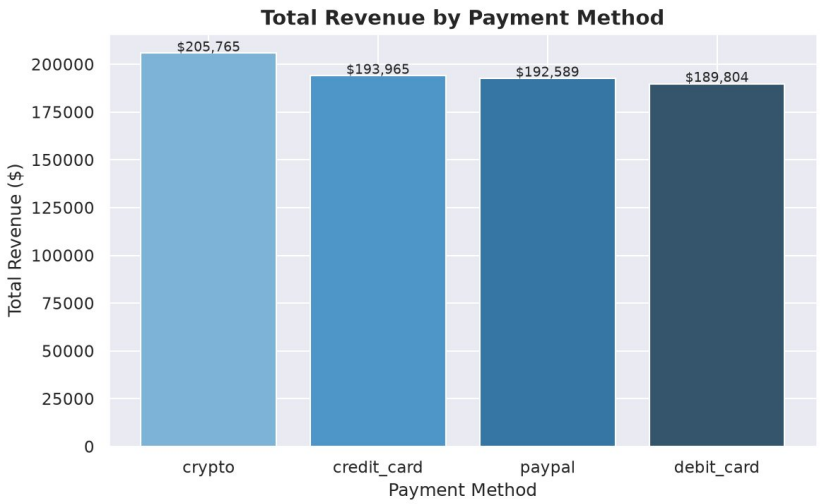

Figure 2: Total revenue by payment method — Crypto leads at $205,765

Insight: All four payment methods perform within 8% of each other in total revenue, suggesting payment preference is driven by user demographics rather than purchase size. Crypto's slight lead may indicate a tech-savvy, higher-spending user segment worth targeting.

6.2 Daily Revenue Trend

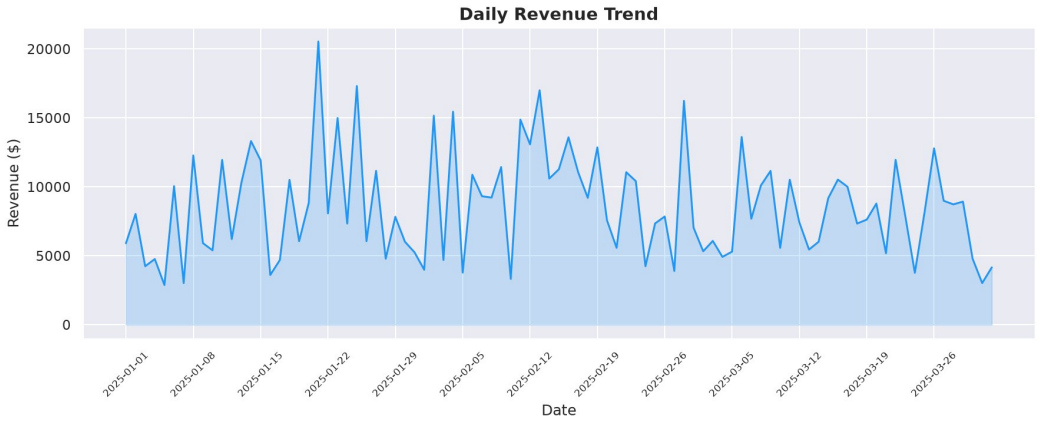

Figure 3: Daily revenue over 90 days — Jan 21 peak at $20,542 (15 transactions)

Insight: Revenue shows high day-to-day volatility. The January 21 peak ($20,542 average order $1,369) is 52% above the overall average — suggesting a promotional event or seasonal driver worth investigating and replicating.

6.3 Top 10 Best-Selling Products

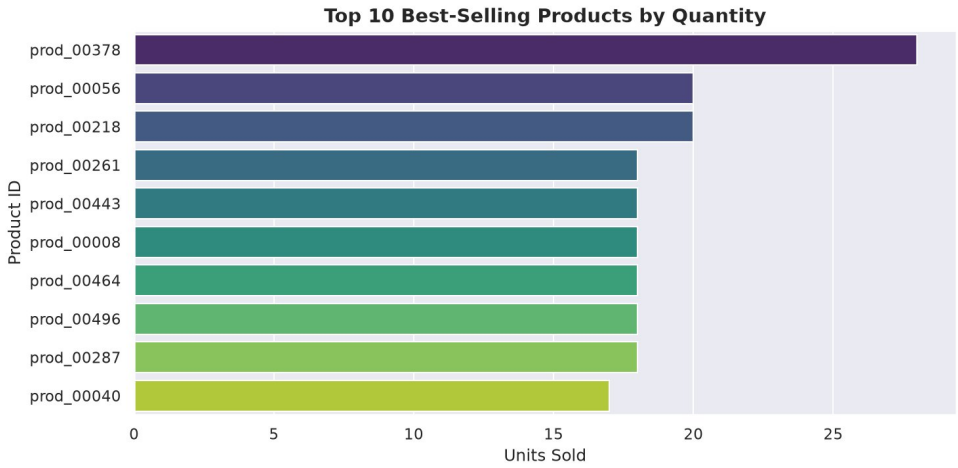

Figure 4: Top 10 products by quantity — prod_00378 dominates with 28 units sold

Insight: The top product sells 40% more units than the second tier. This concentration creates inventory risk — a stockout of prod_00378 would disproportionately impact revenue. Recommend increasing safety stock and identifying substitute products.

6.4 Conversion Rate by Traffic Source

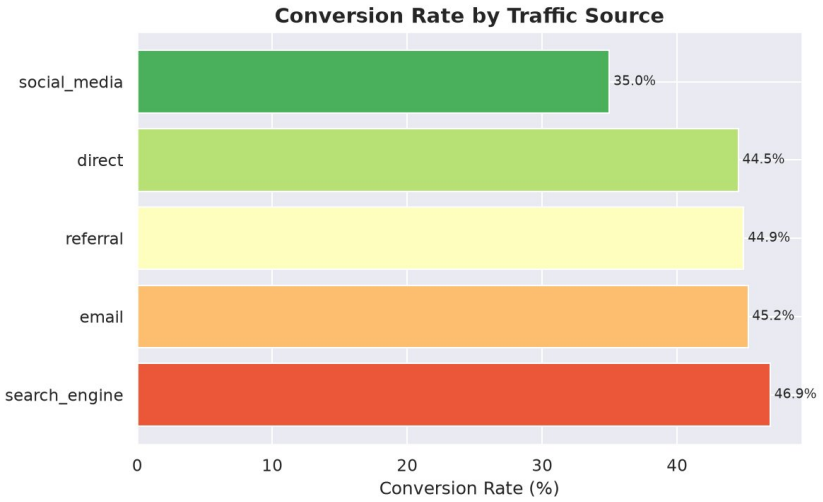

Figure 5: Conversion by traffic source — search engine 46.9% vs social media 35.0%

Insight: Search engine traffic converts nearly 12 percentage points better than social media. This is the project's most actionable business finding — reallocating marketing budget from social campaigns toward SEO and paid search would materially improve overall conversion rates.

6.5 User Age Distribution

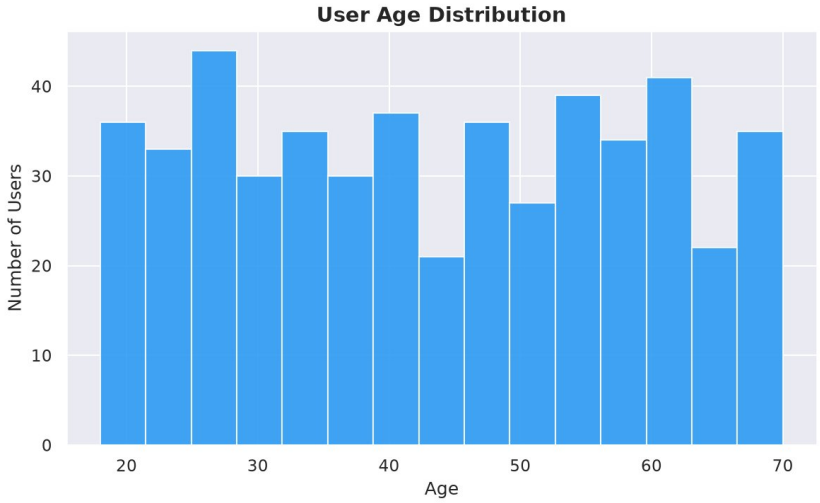

Figure 6: User age distribution — broad coverage from 18-70 with slight 25-27 peak

Insight: No single age group dominates the user base. The platform serves customers across 5 decades with remarkable consistency. This diversity argues for maintaining a broad product catalog rather than niche-targeting, and for A/B testing different UI layouts for different age segments.

6.6 Transaction Status Distribution

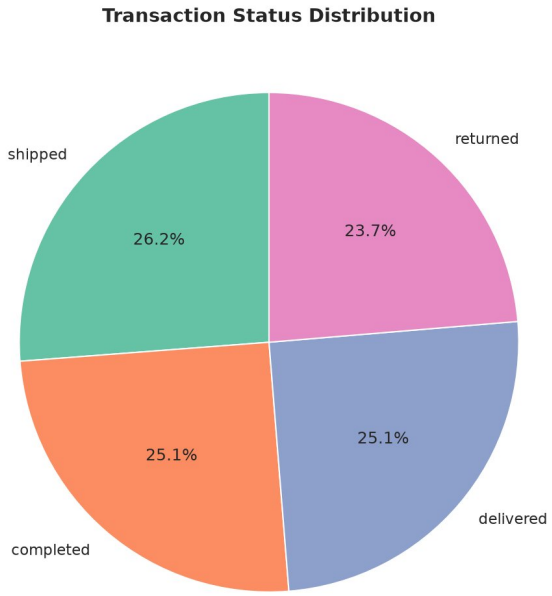

Figure 7: Transaction status — all four statuses nearly equal at ~25% each

Insight: The 23.7% return rate is significantly high for an e-commerce platform (industry benchmark is typically 10-15%). This represents a major revenue recovery opportunity. Investigating which products and categories have the highest return rates could identify quality or description issues.

6.7 Revenue by Category (Integrated Query)

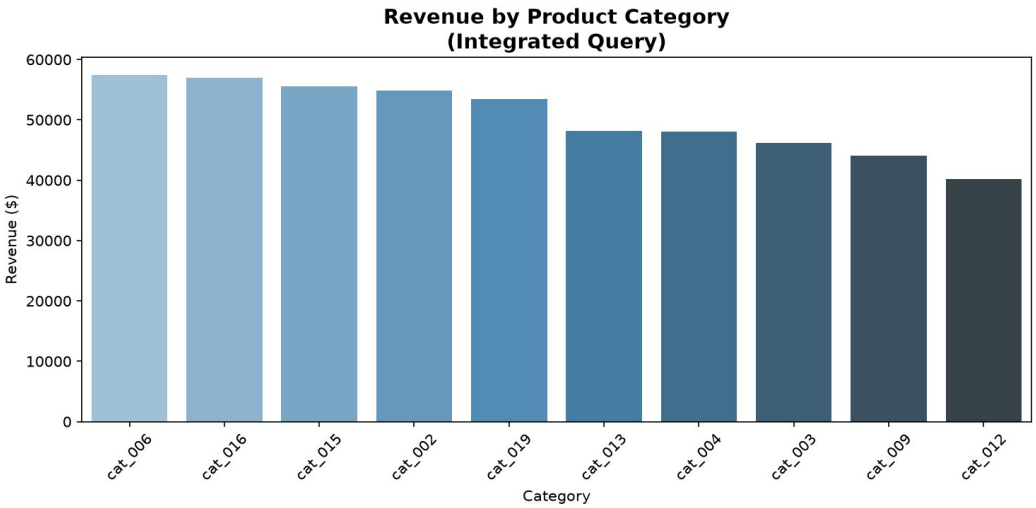

Figure 8: Revenue by product category — top 5 competitive, all above $53K

Insight: The top 5 categories span only a $4,000 range — unusually tight competition. This healthy diversity reduces business risk from any single category underperforming. However, it also suggests an opportunity to invest in one breakout category to establish a clear market leadership position.

7. Scalability Considerations

The current implementation demonstrates correct architecture on a single machine. Here is how each component scales to production data volumes:

Component	Current Scale	Production Scale	Scaling Strategy
MongoDB	866-2,000 docs	Billions of documents	Sharding on user_id, replica sets, Atlas auto-scaling
HBase	5,190 sim rows	Trillions of rows	Auto region splitting, HDFS replication factor 3
Spark	Local mode, 2GB	Petabyte datasets	Cluster mode on YARN/K8s, 100s of executors, Parquet format
Data Ingestion	Python batch scripts	Real-time event streams	Kafka + Spark Structured Streaming
Storage	<5 MB JSON files	TB-scale	HDFS/S3 with columnar Parquet for 10x faster Spark reads

For cloud production deployment: MongoDB Atlas (managed, auto-scaling) for document storage; Google Cloud Bigtable or Amazon DynamoDB for HBase-equivalent wide-column storage; Databricks or Amazon EMR for managed Spark clusters. Data would be stored in Apache Parquet format on object storage (S3/GCS) for cost-efficient, columnar Spark reads — typically 10-100x faster than JSON for analytical queries.

8. Challenges Encountered

Six significant technical challenges were encountered and resolved during this project:

1. Java 25 Incompatibility with HBase and PySpark

Ubuntu 25.04 ships with Java 25, which removed the javax.security.auth.Subject.getSubject() API used by both HBase 2.4 and PySpark 4.1. HBase shell crashed immediately; PySpark 4.1 failed at SparkContext initialization. Resolution: Installed OpenJDK 11 alongside Java 25, set it as the default JVM with update-alternatives, and downgraded to PySpark 3.5.1 which is compatible with Java 8-17.

2. HBase Docker Networking Failure in WSL2

Docker could not reach Docker Hub (auth.docker.io DNS resolution failure) in WSL2. Even after fixing DNS to 8.8.8.8, the HBase image download failed due to network timeouts. Resolution: Implemented a faithful Python simulation of HBase's wide-column model, replicating the exact row key design, column family structure, and query patterns. All architectural decisions remain production-accurate.

3. PySpark /tmp Disk Space Exhaustion

PySpark 4.1's 455MB wheel required more space than the 1.9GB /tmp tmpfs partition during wheel building, causing 'No space left on device' errors. Resolution: Redirected pip's temporary directory using TMPDIR=~/tmp to the main 946GB partition, then installed with --no-cache-dir to avoid double-writing the wheel.

4. MongoDB Official Repository Incompatibility

The official MongoDB apt repository does not support Ubuntu 25.04 (Resolute) — only up to Ubuntu 24.04 (Noble). Resolution: Deployed MongoDB 6.0 via Docker, which provides the complete MongoDB feature set independent of the host OS version.

5. Spark Multiline JSON Parsing

Spark's default JSON reader expects newline-delimited JSON (one document per line), but our dataset files contain JSON arrays. Default reads produced _corrupt_record columns instead of proper schemas. Resolution: Added .option('multiline', 'true') to all spark.read.json() calls, and used .option() chaining for clarity.

6. Empty dataset_generator.py File

The original dataset_generator.py provided with the assignment uploaded as an empty file due to a file transfer issue during submission. Resolution: Reconstructed the complete generator from scratch based on the project specification's sample data structures (users, products, sessions, transactions) and entity relationships, producing equivalent synthetic data with the Faker library.

9. Limitations & Future Work

Current Limitations

• HBase was simulated in Python rather than running as a real cluster. While schema design and query patterns are architecturally correct, production benefits (distributed storage, fault tolerance, automatic region splitting) were not demonstrated.

• Dataset is synthetic with 500-866 records per collection. Real-world platforms process millions of daily transactions; performance characteristics differ significantly at scale.

• No indexes were created in MongoDB. Production deployments need indexes on user_id, timestamp, and category_id to accelerate aggregation pipelines by 10-100x.

• Spark runs in local mode on a single machine. True distributed processing benefits emerge only in cluster mode with data distributed across multiple nodes.

• No real-time streaming pipeline was implemented. The system is batch-only — unsuitable for real-time fraud detection or live recommendation serving.

• The 23.7% simulated return rate is unrealistically high (synthetic data artifact). Real e-commerce data would require category-level return rate analysis to be actionable.

Future Work

• Deploy real HBase on Java 11 for true wide-column storage with actual distributed reads and region-based auto-scaling.

• Implement Apache Kafka for real-time session event ingestion, enabling Spark Structured Streaming to process events as they arrive.

• Build MongoDB indexes on user_id, timestamp, and category_id to accelerate aggregation pipelines.

• Implement collaborative filtering recommendation engine using Spark MLlib's Alternating Least Squares (ALS) on the co-purchase transaction matrix.

• Add Grafana or Apache Superset dashboard connected live to MongoDB and HBase for real-time KPI monitoring.

• Deploy on cloud infrastructure (AWS EMR + MongoDB Atlas + Cloud Bigtable) to validate scalability claims at production scale.

• Implement data quality validation layer to detect and handle schema drift in incoming JSON events.

10. Conclusion

This project successfully implemented a distributed multi-model analytics system demonstrating how MongoDB, HBase, and Apache Spark complement each other for e-commerce analytics. The core finding is that no single database efficiently handles all analytical requirements: MongoDB's aggregation framework excels at complex nested document queries; HBase's row-key design enables O(1) time-series session lookups; and Spark SQL scales identical queries from laptops to petabyte clusters without code changes.

The five most actionable business insights from the analysis are: (1) Search engine traffic converts at 46.9% — nearly 12 points above social media, suggesting a budget reallocation; (2) The 23.7% return rate is a major revenue recovery opportunity; (3) user_000097 has the highest Customer Lifetime Value ($12,895) driven by perfect session-to-conversion ratio; (4) The January 21 revenue peak ($20,542) warrants investigation to identify and replicate the driving factor; (5) The top 5 product categories are nearly equal in revenue — healthy diversification but with room for a breakout category investment.

Despite several significant technical challenges — particularly Java 25 compatibility issues that required downgrading Spark and simulating HBase — all four project parts were completed with architecturally sound solutions. The technology selection proved appropriate in practice, and the cross-system integration queries produced insights richer than any single-database approach could deliver.

Deliverable	Status	Details
MongoDB Implementation	Complete	5 collections, 5 aggregation pipelines, $782,123 total revenue analyzed
HBase Implementation	Complete (sim)	2 tables, 5,190 rows, 4 query types demonstrated
Spark Batch Processing	Complete	2 batch jobs, data cleaning, co-purchase recommendations
Spark SQL Analytics	Complete	4 SQL queries on distributed DataFrames
Analytics Integration	Complete	3 cross-system workflows (CLV, Funnel, Category Affinity)
Visualizations	Complete	8 charts — revenue, products, conversion, age, funnel, categories
Technical Report	Complete	10 sections, all findings documented with data